

Dados Desbalanceados

Quando 99% de acurácia significa que o modelo não aprendeu nada

Fraude, churn, falha de equipamento e diagnóstico de doenças raras compartilham uma característica: a classe que interessa é minoria — às vezes bem menos de 1% dos dados. Um modelo que sempre prevê "não" acerta quase sempre e não serve para nada. Este material cobre por que acurácia falha nesse regime, as técnicas de reamostragem e pesos de classe para corrigir o treino, e o ajuste de limiar de decisão pelo custo real de cada tipo de erro.

Nível: Intermediário — encerra o tema de Preparação de Dados

Pre-requisitos: Todos os módulos anteriores deste tema

Carga sugerida: 8 a 10 horas (leitura + 3 notebooks)

Notebooks: 01-o-problema-do-desbalanceamento · 02-tecnicas-de-reamostragem · 03-limiar-e-custo · 99-exercicios

Autoria: Material do curso de ML & DL

Versao: 1.0

Sumário

1. Por que este módulo existe	3
1.1 O que você vai conseguir fazer ao final	3
2. Por que acurácia falha	4
3. Reamostragem: mudando a distribuição de treino	5
3.1 Undersampling: descartar exemplos da classe majoritária	5
3.2 Oversampling aleatório: duplicar exemplos da classe minoritária	5
3.3 SMOTE: sintetizando novos exemplos por interpolação	5
4. Pesos de classe: ajustando a função de perda, não os dados	7
5. Ajuste de limiar: a alavanca mais barata e mais esquecida	8
6. Erros que custam caro — checklist	9
7. Para ir além	10

1. Por que este módulo existe

Um modelo de detecção de fraude que prevê "não é fraude" para **toda** transação acerta 99,8% das vezes, se a fraude real ocorrer em 0,2% dos casos. Essa acurácia de 99,8% é ao mesmo tempo tecnicamente verdadeira e completamente inútil — o modelo nunca identifica uma única fraude. Esse é o problema central deste módulo: quando as classes são desbalanceadas, a métrica mais intuitiva (acurácia) se torna a métrica mais enganosa, e o algoritmo de treino, otimizando por padrão a mesma métrica implícita, aprende a ignorar a classe rara.

ANALOGIA

Treinar um modelo em dados desbalanceados sem nenhum ajuste é como preparar alguém para reconhecer uma doença rara mostrando 999 fotos de pessoas saudáveis para cada 1 foto de alguém doente. A estratégia que minimiza erros nesse treino é simplesmente **nunca diagnosticar a doença** — e é exatamente essa estratégia inútil que o algoritmo de otimização padrão vai encontrar, porque ela de fato minimiza o erro médio.

1.1 O que você vai conseguir fazer ao final

- Explicar por que acurácia (e até, com ressalvas, a função de perda padrão) falha em dados desbalanceados.
 - Aplicar reamostragem (over, under, SMOTE) e pesos de classe, e saber o trade-off de cada abordagem.
 - Ajustar o limiar de decisão a partir do custo real de negócio de cada tipo de erro, em vez de usar 0,5 por padrão.
 - Avaliar um modelo de classe rara sem se enganar com a distribuição de treino usada internamente.
-

2. Por que acurácia falha

$$\text{acurácia} = \frac{\text{acertos}}{\text{total}}$$

Com 99,8% de exemplos negativos, um classificador que sempre prevê "negativo" tem 99,8% de acurácia sem nenhum poder discriminativo. A acurácia trata um falso negativo (perder uma fraude real) e um falso positivo (marcar uma transação legítima como suspeita) como **igualmente ruins e igualmente raros de acontecer**, quando nem uma coisa nem outra costuma ser verdade.

NO MERCADO

O antídoto imediato para essa armadilha específica é métrica, não técnica de treino: reportar precisão, recall, F1 e PR-AUC (não ROC-AUC sozinha) no lugar de acurácia. O tema 6 (Avaliação e Validação) aprofunda cada uma dessas métricas — este módulo assume que você vai medir certo, e foca em como **treinar** melhor dado o desbalanceamento.

ARMADILHA COMUM

ROC-AUC é enganosamente estável sob desbalanceamento severo — ela pode parecer "boa" mesmo quando o modelo tem pouquíssima precisão na classe rara, porque a taxa de falso positivo (o eixo x da curva ROC) é calculada sobre a classe majoritária, que é grande e estável. PR-AUC (precisão-recall) é mais sensível ao desempenho real na classe minoritária — a métrica preferida quando a classe positiva é rara e é a que importa.

3. Reamostragem: mudando a distribuição de treino

3.1 Undersampling: descartar exemplos da classe majoritária

Reduz a classe majoritária até equilibrar (ou aproximar) a proporção com a minoritária. Simples e rápido, mas **descarta informação real** — com desbalanceamento severo (fraude a 0,2%), undersampling puro jogaria fora a maior parte dos dados disponíveis.

3.2 Oversampling aleatório: duplicar exemplos da classe minoritária

Repete exemplos da classe rara até equilibrar as proporções. Não perde informação da classe majoritária, mas **duplicatas exatas aumentam o risco de overfitting** — o modelo pode memorizar os poucos exemplos raros repetidos em vez de generalizar o padrão que eles representam.

3.3 SMOTE: sintetizando novos exemplos por interpolação

SMOTE (*Synthetic Minority Oversampling Technique*) não duplica — cria exemplos **novos** e sintéticos da classe minoritária, interpolando entre um ponto real e um de seus vizinhos mais próximos (também da classe minoritária):

FORMULARIO

Para um ponto minoritário x_i e um vizinho minoritário x_{viz} escolhido entre os k mais próximos: gera-se um novo ponto sintético $x_{novo} = x_i + \lambda(x_{viz} - x_i)$, com λ sorteado uniformemente em $[0, 1]$ — um ponto em algum lugar do segmento de reta entre os dois.

ARMADILHA COMUM

SMOTE opera no espaço de **features contínuas** e assume que interpolar entre dois pontos produz um exemplo plausível da classe — o que quebra com features categóricas (a "interpolação" entre duas categorias não tem significado) e pode gerar pontos sintéticos em regiões onde, na verdade, as classes se sobrepõem (perto da fronteira de decisão real), piorando a separabilidade em vez de ajudar. Variantes como o Borderline-SMOTE tentam mitigar isso focando a interpolação perto da fronteira de decisão de forma mais cuidadosa.

ARMADILHA COMUM

Reamostragem — de qualquer tipo — deve acontecer **só no conjunto de treino**, nunca no conjunto de validação/teste. Testar contra dados reamostrados mede o desempenho num mundo artificial que não existe em produção; o teste precisa refletir a proporção real que o modelo vai enfrentar. Isso é o mesmo princípio de vazamento do módulo 4: qualquer transformação que depende dos dados deve respeitar a fronteira treino/teste — inclusive dentro de cada fold de validação cruzada.

4. Pesos de classe: ajustando a função de perda, não os dados

Em vez de mudar os dados, muda-se a **penalidade** de errar cada classe. A maioria dos classificadores do `sklearn` aceita `class_weight="balanced"`, que pondera o erro de cada classe pelo inverso da sua frequência — errar um exemplo raro custa proporcionalmente mais que errar um comum.

NOTA

Pesos de classe e reamostragem atacam o mesmo problema por vias diferentes, e frequentemente produzem resultados parecidos. A vantagem dos pesos: não alteram o dataset (nenhum ponto sintético, nenhuma duplicata), então não há risco de overfitting por repetição, e o custo computacional não aumenta (o dataset não cresce). A vantagem da reamostragem: funciona com qualquer algoritmo, mesmo os que não aceitam pesos por amostra nativamente.

5. Ajuste de limiar: a alavanca mais barata e mais esquecida

Um classificador probabilístico produz uma probabilidade; a conversão para "sim/não" usando o limiar padrão de 0,5 é uma **escolha arbitrária**, não uma lei. Ajustar esse limiar é, com frequência, mais eficaz e muito mais barato do que reamostrar ou treinar de novo.

FORMULARIO

Dado um custo C_{FP} para falso positivo e C_{FN} para falso negativo, o limiar ótimo (sob suposições de calibração razoável) se desloca na direção que reflete a assimetria de custo: quanto mais caro um falso negativo em relação a um falso positivo, **mais baixo** deve ser o limiar para classificar como positivo — o modelo passa a soar o alarme com menos evidência, porque o custo de deixar passar é maior que o custo de investigar à toa.

NO MERCADO

Em detecção de fraude, um falso negativo (fraude não detectada) custa o valor da fraude; um falso positivo (transação legítima bloqueada) custa uma experiência ruim do cliente e talvez uma ligação de suporte — quase sempre bem mais barato. Isso justifica limiares bem abaixo de 0,5. Em triagem médica de uma doença grave, a mesma lógica se aplica com ainda mais força: perder um caso real (falso negativo) tende a ser muito mais custoso que investigar um caso saudável (falso positivo).

A curva precisão-recall, variando o limiar, mostra exatamente esse trade-off: um limiar mais baixo aumenta recall (captura mais casos positivos reais) às custas de precisão (mais falsos positivos), e vice-versa. Escolher o ponto certo da curva é uma decisão de negócio, não estatística.

6. Erros que custam caro — checklist

- Reportar acurácia como métrica principal em um problema com classe rara.
- Reamostrar o conjunto de teste/validação, medindo desempenho num cenário artificial que não existe em produção.
- Usar SMOTE ingenuamente em features categóricas, gerando pontos sintéticos sem significado.
- Deixar o limiar de decisão em 0,5 por padrão sem considerar o custo real de cada tipo de erro.
- Confiar em ROC-AUC como única métrica quando a classe positiva é rara — prefira PR-AUC, precisão, recall e F1.
- Combinar reamostragem agressiva com um limiar padrão de 0,5 sem recalibrar — a probabilidade de saída deixa de refletir a proporção real depois do oversampling, e um limiar "ingênuo" pode ficar sistematicamente errado.

7. Para ir além

- Chawla et al. (2002), o artigo original do SMOTE.
- He & Garcia, *Learning from Imbalanced Data* — o levantamento mais citado da área, cobrindo reamostragem, pesos e métricas.
- Documentação do [imbalanced-learn](#) — implementações de referência de SMOTE e variantes (embora este módulo construa a lógica do zero, por clareza pedagógica).
- Provost & Fawcett, *Data Science for Business*, capítulo sobre matrizes de custo — a formalização do raciocínio de custo assimétrico usado neste módulo.